

Trabajo Práctico N.º 7

CRUD Nivel 3 — Sistema de Biblioteca (agregaciones, transacciones y consultas integradoras)

Materia	Bases de Datos Avanzadas
Tema	Aggregation Framework, \$lookup, índices de texto y transacciones multi-documento
Modalidad de trabajo	Individual o en parejas
Herramientas	MongoDB Community / MongoDB Atlas, mongosh, MongoDB Compass

1. Introducción

Este es el tercer y último práctico de la secuencia iniciada en el TP N.º 5, y el de mayor complejidad de toda la cursada. Sobre el mismo sistema de biblioteca (colecciones libros, socios y prestamos), se incorporan las herramientas más potentes del CRUD de MongoDB: el Aggregation Framework para construir reportes y estadísticas, el operador \$lookup para resolver "joins" entre colecciones relacionadas, los índices de texto para búsquedas por contenido, y las transacciones multi-documento para garantizar la atomicidad de operaciones que afectan a más de una colección a la vez.

Se asume que las colecciones libros, socios y prestamos del TP N.º 6 ya existen y tienen datos cargados.

2. Objetivos de aprendizaje

- Construir pipelines de agregación combinando las etapas \$match, \$group, \$project, \$sort y \$limit.
- Resolver relaciones entre colecciones mediante \$lookup, emulando un join del modelo relacional.
- Utilizar \$unwind para "desanidar" arrays dentro de un pipeline.
- Crear y utilizar un índice de texto para búsquedas de contenido libre.
- Comprender el propósito de las transacciones multi-documento y ejecutar una transacción simple con sesión.
- Diseñar e implementar un caso integrador que combine todo lo anterior en un reporte final.

3. Marco teórico

3.1 El Aggregation Framework

El método aggregate() ejecuta un pipeline: una secuencia ordenada de etapas (stages), donde la salida de una etapa es la entrada de la siguiente, de forma análoga a una tubería Unix. A diferencia de find(), que solo filtra y proyecta, el pipeline permite agrupar, calcular estadísticas, transformar la forma de los documentos y combinar colecciones.

Etapa	Propósito
\$match	Filtra documentos (misma sintaxis que el filtro de find()).
\$group	Agrupar documentos por un campo (o expresión) y calcula acumuladores: \$sum, \$avg, \$min, \$max, \$push, entre otros.
\$project	Da forma al documento de salida: incluye, excluye o calcula nuevos campos.
\$sort / \$limit	Ordenan y limitan los resultados, igual que en find().
\$unwind	Descompone un array en un documento independiente por cada uno de sus elementos.
\$lookup	Realiza una combinación ("join") con otra colección según un campo local y uno foráneo.

Ejemplo: cantidad de libros por género, ordenados de mayor a menor:

```
db.libros.aggregate([
  { $group: { _id: "$genero", cantidad: { $sum: 1 } } },
  { $sort: { cantidad: -1 } }
])
```

3.2 \$lookup: combinando colecciones relacionadas

\$lookup permite, para cada documento de la colección "local", buscar en otra colección ("foránea") los documentos cuyo campo indicado coincida, y agregarlos como un array embebido en el resultado. Es la herramienta clave para leer datos que fueron modelados de forma referenciada (como se decidió para prestamos en el TP N.º 6).

```
db.prestamos.aggregate([
  {
    $lookup: {
      from: "libros",
      localField: "libroId",
      foreignField: "_id",
      as: "libro"
    }
  },
  { $unwind: "$libro" },
  {
    $project: {
      _id: 0,
      libro: "$libro.titulo",
      fechaPrestamo: 1,
      devuelto: 1
    }
  }
])
```

Nótese el uso de \$unwind: como \$lookup siempre agrega un array (aun si hay una única coincidencia), es habitual "desanidarlos" con \$unwind para trabajar con el subdocumento directamente en lugar de con un array de un elemento.

3.3 Índices de texto

Un índice de texto (text index) permite realizar búsquedas de palabras dentro de campos de tipo string, similares a un buscador, en lugar de comparaciones exactas. Solo puede existir un índice de texto por colección, aunque puede abarcar varios campos.

```
db.libros.createIndex({ titulo: "text", autor: "text" })

// Búsqueda de texto libre
db.libros.find({ $text: { $search: "garcía soledad" } })
```

3.4 Transacciones multi-documento

Una transacción agrupa varias operaciones de escritura (potencialmente sobre distintas colecciones) para que se ejecuten de forma atómica: o se confirman todas (commit), o se revierten todas (abort) si ocurre un error. Son imprescindibles cuando una operación de negocio requiere modificar coherentemente más de un documento a la vez, como registrar un préstamo (que debe, a la vez, crear el documento de préstamo y decrementar el stock del libro).

```
const session = db.getMongo().startSession();
session.startTransaction();
try {
  const libros = session.getDatabase("biblioteca").libros;
  const prestamos = session.getDatabase("biblioteca").prestamos;

  libros.updateOne({ _id: "L-0003" }, { $inc: { ejemplaresDisponibles: -1 } });
  prestamos.insertOne({
    _id: "PR-0011", socioId: "S-002", libroId: "L-0003",
    fechaPrestamo: "2026-05-10", fechaDevolucion: null, devuelto: false
  });

  session.commitTransaction();
} catch (error) {
  session.abortTransaction();
  print("Transacción revertida: " + error);
} finally {
  session.endSession();
}
```

Requisito técnico

Las transacciones multi-documento requieren que el servidor esté configurado como replica set (incluso de un único nodo) o como clúster sharded; no funcionan sobre una instancia standalone simple. Si su entorno no lo soporta, documente el código de la transacción y explique conceptualmente su funcionamiento, señalando en qué punto fallaría en un standalone.

4. Consignas de trabajo

Ejercicio 1 — Estadísticas por género con \$group

Construya un pipeline que devuelva, para cada género, la cantidad de libros y el promedio de ejemplaresDisponibles, ordenado de mayor a menor cantidad de libros.

Ejercicio 2 — Reporte de préstamos por socio con \$lookup

Construya un pipeline sobre prestamos que combine, mediante \$lookup, los datos del socio (nombre) y del libro (título), y muestre un listado con: nombre del socio, título del libro, fecha de préstamo y si fue devuelto. Utilice \$unwind sobre ambos lookups.

Ejercicio 3 — Ranking de socios más activos

Construya un pipeline sobre prestamos que agrupe por socioid, cuente la cantidad de préstamos de cada socio (\$sum), y luego combine (\$lookup) con socios para mostrar el nombre en lugar del identificador. Muestre solo el top 3 mediante \$sort y \$limit.

Ejercicio 4 — Libros nunca prestados

Diseñe un pipeline (puede usar \$lookup seguido de un \$match sobre el tamaño del array resultante, con \$expr y \$size, o la variante que considere más clara) que liste los libros del catálogo que nunca registraron un préstamo.

Justifique por escrito la estrategia elegida.

Ejercicio 5 — Búsqueda de texto libre

Cree un índice de texto sobre titulo y autor en la colección libros. Ejecute al menos tres búsquedas distintas con \$text/\$search (por autor, por una palabra del título, y por una combinación de ambos) y documente los resultados obtenidos en cada caso.

Ejercicio 6 — Transacción de préstamo

Implemente, con sesión y transacción, la operación de "otorgar un préstamo": debe decrementar en 1 el campo ejemplaresDisponibles del libro correspondiente e insertar el nuevo documento en prestamos, de forma atómica. Simule además un caso de error controlado (por ejemplo, intentando prestar un libro sin ejemplares disponibles) y muestre que la transacción se revierte sin dejar cambios parciales.

Ejercicio 7 — Caso integrador final

Diseñe y documente un reporte final de la biblioteca que combine, en un único pipeline o en una secuencia breve de consultas, al menos: una etapa de \$match, una de \$group, un \$lookup y una proyección final (\$project) con nombres de campos en español, claros para un usuario no técnico.

El reporte debe responder una pregunta de negocio real definida por usted (por ejemplo: "¿cuáles son los 5 géneros más solicitados en préstamo durante el último semestre?"). Redacte la pregunta elegida, el pipeline completo y una interpretación de los resultados obtenidos en 5 a 10 líneas.

Cierre de la secuencia

Este caso integrador cierra el recorrido iniciado en el TP N.º 1: del diseño conceptual de un documento JSON a un sistema completo capaz de responder preguntas de negocio mediante el Aggregation Framework de MongoDB.

5. Forma de entrega

- Archivo o script con todos los pipelines y comandos ejecutados, en el orden de los ejercicios.
- Documento (PDF o Word) con las salidas de cada consulta y las explicaciones e interpretaciones solicitadas.
- Plazo de entrega y vía de entrega: a definir por la cátedra en el aula virtual.

6. Criterios de evaluación

Criterio	Descripción	Puntos
Pipelines de agregación (\$group/\$project)	Uso correcto de etapas y acumuladores; resultados coherentes con los datos.	20
\$lookup y \$unwind	Combinación correcta de colecciones relacionadas y desanidado adecuado.	25
Índice de texto	Índice creado correctamente y búsquedas \$text bien resueltas.	15
Transacción multi-documento	Implementación correcta de la sesión/transacción, incluyendo el caso de error y su reversión.	20
Caso integrador final	Pregunta de negocio relevante, pipeline bien construido e interpretación de resultados clara y fundamentada.	20
Total		100